

Directx12 미니 프로젝트



깃허브 주소

<https://github.com/crescendo103/studydirectx>



실행 영상

<https://youtu.be/1qtICc5XPiU>

목차

게임소개
클래스 구조도
쉐이더 코드

구현한 이모저모
-보석 위치를 알려주는 UI
-계단충돌체 설정
-길 찾기 알고리즘
-실행영상

게임소개



제단



플레이어



몬스터



보석



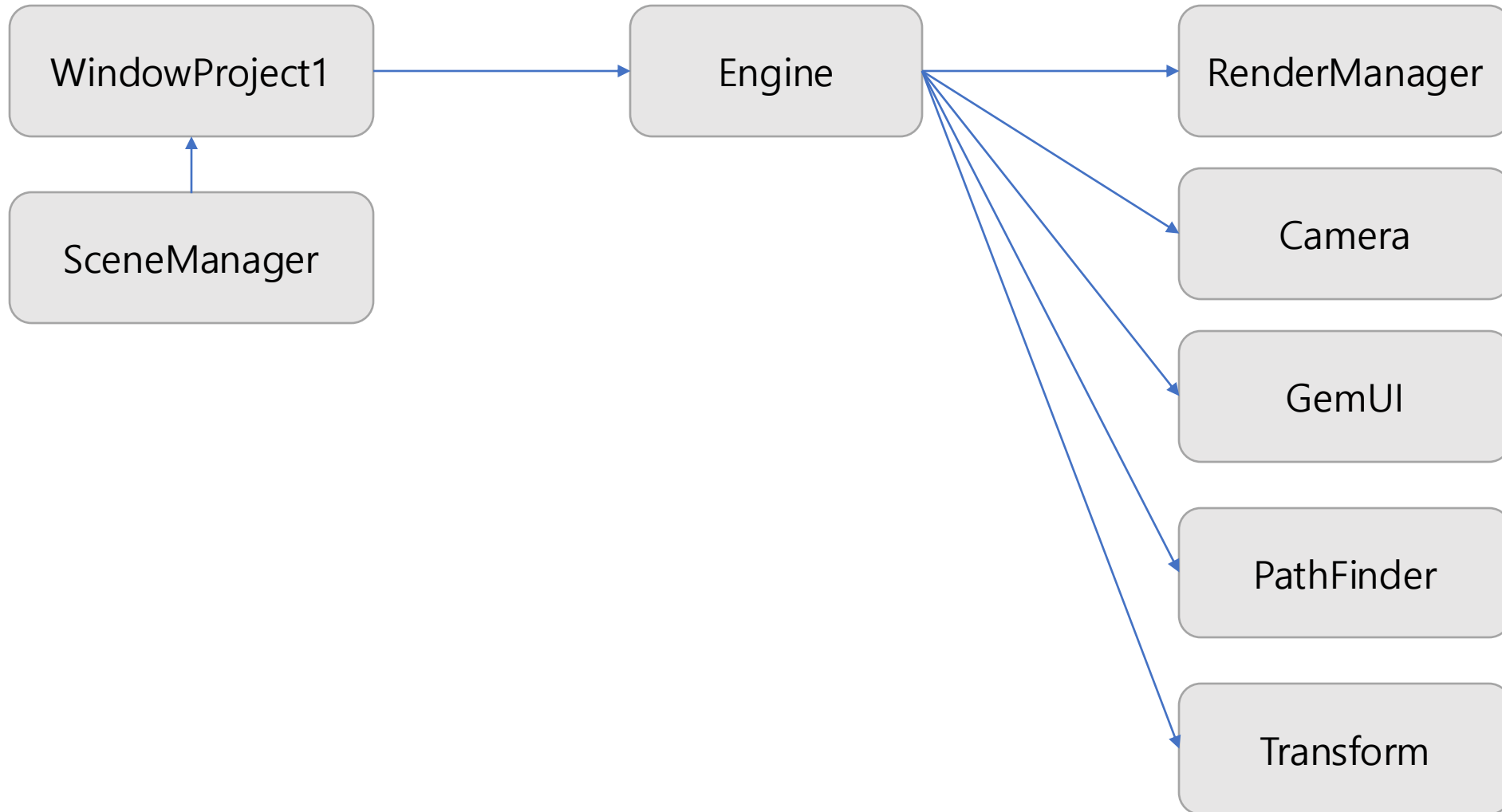
보석



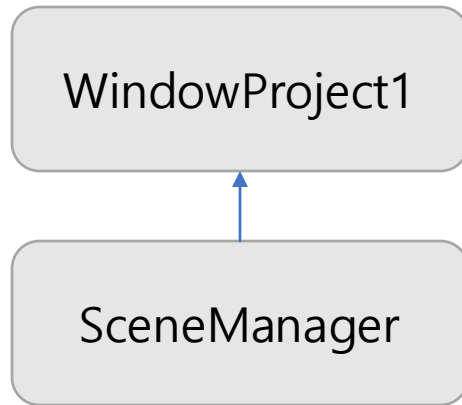
보석

플레이어가 몬스터를 피해 맵에 숨겨진 보석을 모아와서 제단에 두면 클리어되는 형식

클래스 관계도



클래스 관계도



WindowProject1에서는
렌더링 파이프라인에 필요한
장치초기화 및 업데이트를 하고 있습니다.

SceneManager는 장치를 초기화하는 동안
화면에 띄워줄 사진을 관리하고 있습니다.

```
Engine sample(width, height, L"", Global_hwnd);
sample.OnInit();
```

```
// 기본 메시지 루프입니다:
while (true)
{
    if (PeekMessage(&msg, nullptr, 0, 0, PM_REMOVE)) //메세지가 없어도 항상 반환된다
        //메세지가 있다면 true 없다면 false
    {
        if (msg.message == WM_QUIT) {
            break;
        }
        if (!TranslateAccelerator(msg.hwnd, hAccelTable, &msg))
        {
            TranslateMessage(&msg);
            DispatchMessage(&msg);
        }
    }
    else {
        //메세지가 없는동안 호출
        delta moustdelta = ReadMouse();
        sample.OnUpdate(moustdelta.deltaX, moustdelta.deltaY); //업데이트
        if (sample.getshouldMouseCenter()) {
            SetCursorPos(centerPoint.x, centerPoint.y);
        }
    }
}
```

클래스 관계도

Engine

Engine 클래스에서는 장치(Device) 초기화 및 프레임 업데이트를 담당하며, 각 하위 클래스들을 관리하고 UI 렌더링을 위한 별도의 파이프라인과 루트 서술자(Root Descriptor)를 정의하고, globalMaterialOffset 변수를 통해 리소스 오프셋을 계산하여 SetGraphicsRootDescriptorTable 함수를 사용해 셰이더가 해당 리소스에 접근할 수 있도록 구성하였습니다.

```
for (int i = 0; i < main.size(); i++) {  
    Mainset indices = main[i];  
    Subset materialTex = sub[i];  
    if (materialTex.istexture) { //텍스처를 가지고 있는 경우  
        m_commandList->SetGraphicsRootDescriptorTable(1, CD3DX12_GPU_DESCRIPTOR_HANDLE(mCbvSrvHeap->GetGPUDescriptorHandleForHeapStart(), footprint + 1, mCbvSrvDescriptorSize));  
        footprint++;  
        m_commandList->SetGraphicsRootDescriptorTable(3, CD3DX12_GPU_DESCRIPTOR_HANDLE(mCbvSrvHeap->GetGPUDescriptorHandleForHeapStart(), footprint + 1, mCbvSrvDescriptorSize));  
    }  
    else { //텍스처를 가지지 않는 경우 텍스처 자리에 빈 디스크립터 할당  
        m_commandList->SetGraphicsRootDescriptorTable(1, CD3DX12_GPU_DESCRIPTOR_HANDLE(mCbvSrvHeap->GetGPUDescriptorHandleForHeapStart(), footprint + 1, mCbvSrvDescriptorSize));  
        m_commandList->SetGraphicsRootDescriptorTable(3, CD3DX12_GPU_DESCRIPTOR_HANDLE(mCbvSrvHeap->GetGPUDescriptorHandleForHeapStart(), footprint + 1, mCbvSrvDescriptorSize));  
    }  
    m_commandList->IASetIndexBuffer(&indices.indicesBufferView);  
    m_commandList->DrawIndexedInstanced(indices.indicesCount, 1, 0, 0, 0);  
    footprint++;  
}
```

Name	Value
▶ 0 - CBV	res#211
▶ 1 - CBV	res#212
▶ 2 - SRV	res#213
▶ 3 - CBV	res#214
▶ 4 - CBV	res#215
▶ 5 - SRV	res#216
▶ 6 - CBV	res#217
▶ 7 - SRV	res#218
▶ 8 - CBV	res#219
▶ 9 - SRV	res#220
▶ 10 - CBV	res#221
▶ 11 - SRV	res#222
▶ 12 - CBV	res#223
▶ 13 - SRV	res#224

클래스 관계도

재질상수버퍼 생성, 텍스처 버퍼 생성, 인덱스 버퍼 생성
버텍스 버퍼 생성, 물체 회전이동 등등 관리



RenderManager

카메라가 이동하려는 위치에 충돌체가 있을 경우, 해당 위치로
이동하지 못하도록 충돌 판정을 적용하여 이동을 제한



Camera

카메라의 위치(플레이어의 위치)에 따른 보석위치를 알려주는
UI오브젝트 출력여부(bool)를 관리



GemUI

연결리스트를 활용한 몬스터의 이동 관리



PathFinder

몬스터와 플레이어와의 상호작용->범위내에 플레이어 있을
시 플레이어를 쫓아오도록 SRT적용, 범위 밖에 있을 시
PathFinder 클래스를 통한 이동할 좌표 설정



Transform

셰이더 코드



```
// 카메라 위치(gCameraPos)와 픽셀의 월드 좌표(input.worldPos) 간의 벡터 계산
float3 lightDir = normalize(input.worldPos - gCameraPos);
float dist = length(input.worldPos - gCameraPos);
// 거리 기반 감쇠 (range 값은 상황에 맞게 조절)
float range = 20.0f; // 예시: 광원의 영향 범위
if (Lamp == 0.0f) // 랜턴 없을 경우
{
    range = 10.0f;
}
else // 랜턴 있을 경우
{
    range = 20.0f;
}

float attenuation = saturate(1.0f - (dist / range));

// Lambert Diffuse 계산: 뒷면도 보이게 절대값 사용
float NdotL = abs(dot(normal, lightDir));
float3 diffuse1 = texColor * NdotL * attenuation;
// 알파 값은 필요에 따라 조정 (예시에서는 1.0으로 고정)
float3 finalColor = diffuse1;
```

카메라 위치를 광원으로 활용하여 동적인 랜턴 조명 효과를 구현했습니다.
Lamp 상태 값에 따라 조명의 범위를 조절하고, 거리 기반 감쇠(attenuation)를 적용하여
현실적인 광량 감소를 표현하였으며, Lambert 조명을 기반으로 한 양면 조명 처리로
역면에서도 빛을 받도록 했습니다.

쉐이더 코드



```
if (l3 == 1)
{
    float3 pointLightPos = float3(3.9131f, 0.0f, -24.132f); // 추가적인 점광원 위치
    float3 lightDir2 = normalize(input.worldPos - pointLightPos);
    float dist2 = length(input.worldPos - pointLightPos);

    float range2 = 15.0f; // 새로운 점광원의 범위 (적절히 조절 가능)
    float attenuation2 = saturate(1.0f - (dist2 / range2));

    float NdotL2 = abs(dot(normal, lightDir2));
    float3 diffuse2 = texColor * NdotL2 * attenuation2;

    // [3] 두 개의 점광원 조합
    finalColor += diffuse2;
}

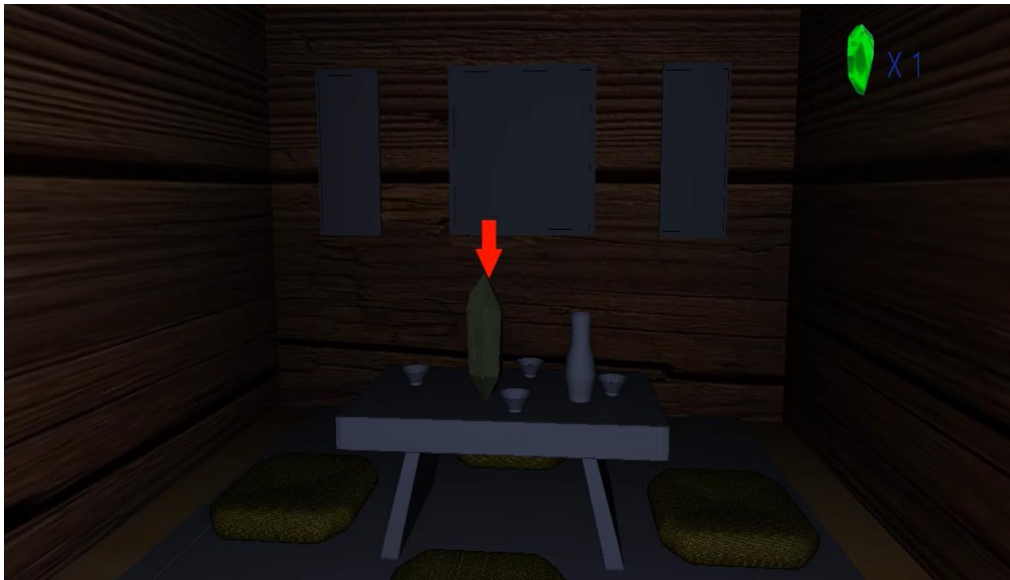
if (l4 == 1)
```

다중 조명 효과 구현을 위해 추가적으로 점광원(Point Light) 생성했습니다. 상수버퍼로 l3를 넘겨주어 광원클릭여부를 관리했습니다. 지정된 위치에 새로운 점광원을 활성화하고, Lambert 모델과 거리 기반 감쇠를 통해 현실적인 광원 세기 감소를 반영했습니다. 계산된 광량(diffuse2)을 기존 조명 결과에 합산하여, 두 개 이상의 광원이 동시에 영향을 주는 장면을 표현할 수 있도록 했습니다.

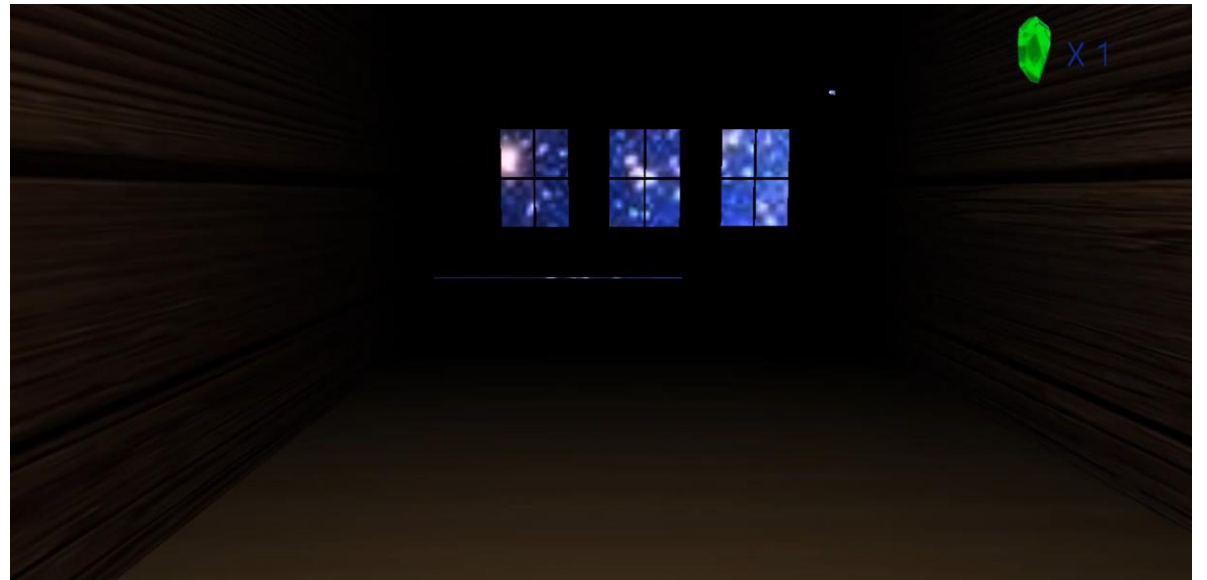
구현한 이모저모

보석 위치 알려주는 UI 오브젝트

기능 요약:



지정위치에서 보석을 응시하고 있을때는
위에 화살표가 나타납니다.



보석을 응시하고 있지 않을때는
위에 화살표가 나타나지 않습니다.

보석 위치 알려주는 UI 오브젝트



updateGem

isLookObject

```
void GemUI::updateGem()
{
    //플레이어가 지정된 좌표안에 있을경우 보석위에 화살표가 표시됨
    if (m_Pos.x > -10.5f && m_Pos.x < -4.5f && m_Pos.z < -28.0f && m_Pos.z > -34.0f) {
        shouldUI = true;
        gem1range = true;
    }

    if (gem1 == false) {
        shouldUI = false;
        gem1range = false;
    }

    }else if (m_Pos.x > -4.0f && m_Pos.x < 1.6f && m_Pos.z < -16.0f && m_Pos.z > -34.0f) {
        shouldUI = true;
        gem2range = true;

        if (gem2 == false) {
            shouldUI = false;
            gem2range = false;
        }

    }else if (m_Pos.x > 2.0f && m_Pos.x < 8.0f && m_Pos.z < -10.0f && m_Pos.z > -20.0f) {
```

updateGem에서 보석이 보이는 위치에 플레이어가 있을경우 화살표를 출력하라는 shouldUI를 True로 해주었습니다.

보석 위치 알려주는 UI 오브젝트



updateGem

isLookObject

```
//플레이어가 물체를 보고있을때 아닐때를 판단하는 함수
bool Engine::isLookObject(const XMFLOAT3& objectPos)
{
    // 카메라 위치와 앞 방향 얻기
    XMFLOAT3 camPos = cam->getMyPos(); // 카메라 위치
    XMMATRIX view = cam->GetView(); //뷰행렬
    XMMATRIX invView = XMMatrixInverse(nullptr, view); //월드좌표계

    XMFLOAT3 forward;
    XMStoreFloat3(&forward, invView.r[2]); // r[2]가 Z축관련 방향

    // 오브젝트까지의 방향 벡터 계산
    XMVECTOR camPosVec = XMLoadFloat3(&camPos);
    XMVECTOR objectPosVec = XMLoadFloat3(&objectPos);
    XMVECTOR toObjectVec = XMVector3Normalize(objectPosVec - camPosVec);

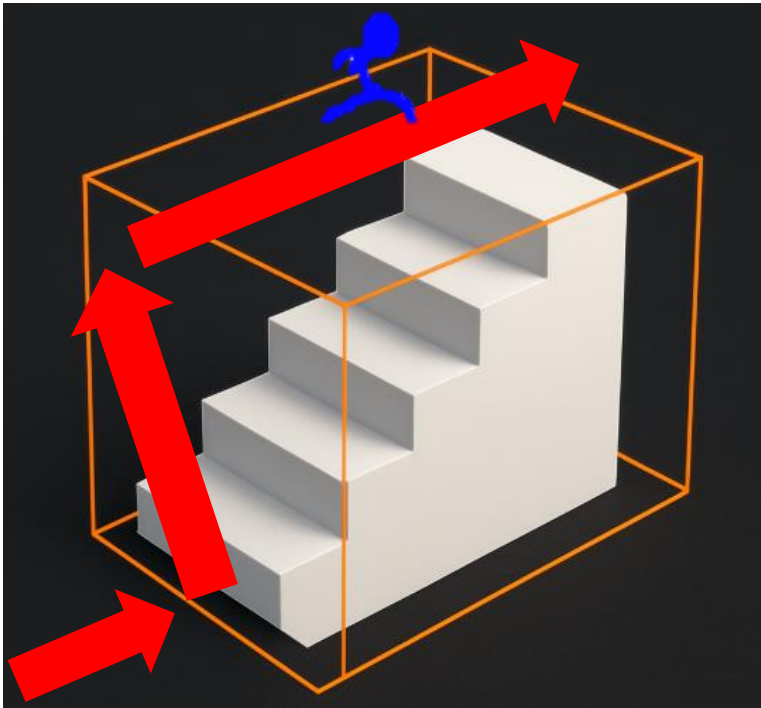
    // 카메라의 forward 벡터
    XMVECTOR camForwardVec = XMVector3Normalize(XMLoadFloat3(&forward));

    // 내적 계산
    float dot = XMVectorGetX(XMVector3Dot(toObjectVec, camForwardVec));

    return dot > 0.0f;
}
```

이후 카메라가 바라보는 방향과 카메라->물체 방향의 내적값이 양수인경우에 다시 shouldUI값을 true로 세팅해주었습니다.

계단 충돌체 설정

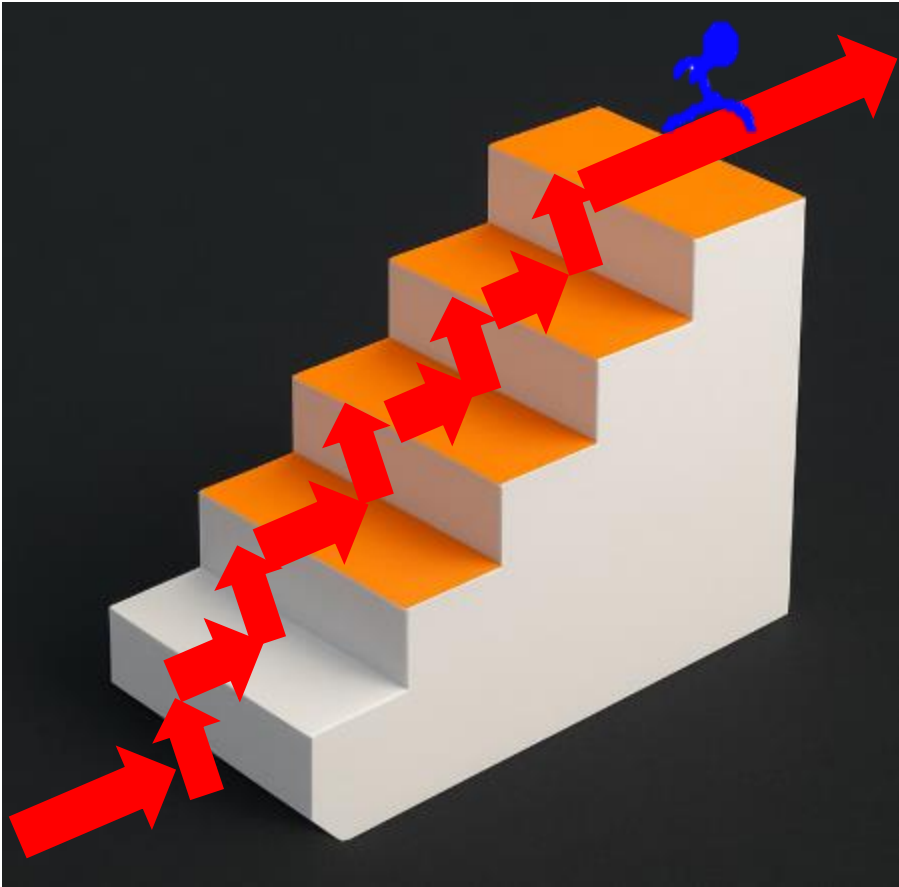


문제

플레이어가 계단을 지날때 충돌체 검사를 통해 boundingbox 위에 있게 됩니다.

기존 boundingbox를 사용하면 왼쪽 그림과 같이 충돌 경계가 설정되어 자연스러운 계단 오르는 연출이 되지않습니다.

계단 충돌체 설정



해결방안

Y축 노멀값이 1에 가깝고 x,z축 노멀값은 0에 가까운 면들만 추출해서 경계상자를 만들어 주었습니다.

계단 충돌체 설정

```
int vn_index = normals.size();  
  
if (std::abs(x) < Diff && std::abs(y - 1.0f) < Diff && std::abs(z) < Diff) {  
    y_like_normal_indices.push_back(vn_index);  
}  
}
```



함수 stairYVertex()를 사용해 설정값(Diff)보다 노멀값(vn)의 x와 z값은 설정값 아래 (0에 가까운 경우) y값은 1에 가까운 면(즉 y축에 거의 평행한 면)의 정점을 face_vertex_coords 벡터에 저장하고

```
//면의 x,y2차원 경계상자와 그때 y좌표 설정  
void Camera::calculateSlope()  
{  
    for (int i = 0; i < stairYPos.size(); i++) {  
        MinMax minmax_face;  
        for (int j = 0; j < stairYPos[i].size(); j++) {  
            Vec3 pos = stairYPos[i][j];  
            minmax_face.anyYpos = pos.y; //면위의 임의의 y값  
        }  
    }  
}
```



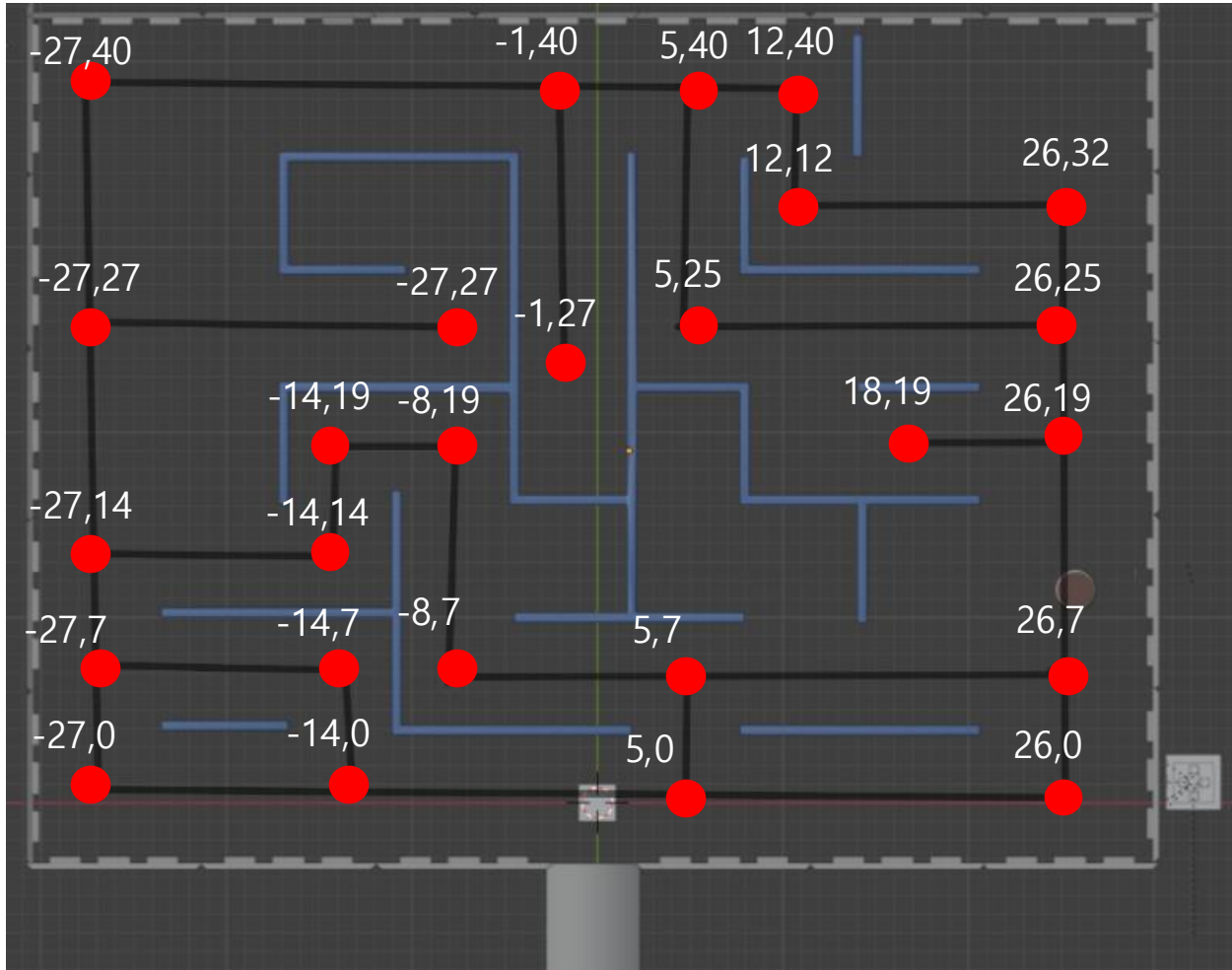
calculateSlope() 함수를 통해 벡터에서 해당 면의 x,z값의 최대 최소값을 y 값은 면 위의 임의의 점(중요하지 않다)으로 MinMax 구조체 값으로 설정하였습니다.

```
//계단에 위치할 경우 y값  
XMFLOAT3 val=myPos;  
for (int i = 0; i < boundingFace.size(); i++) {  
    MinMax candidate = boundingFace[i];  
    if ((myPos.x - candidate.maxX) * (myPos.x - candidate.minX) < 0.0f &&  
        (myPos.z - candidate.maxZ) * (myPos.z - candidate.minZ) < 0.0f) {  
        val.y = candidate.anyYpos + 2.0f;  
    }  
}
```



calculateCamYpos()에서는 플레이어가 계단의 면위의 x,z값 범위에 있을 경우 플레이어의 위치를 면위의 y좌표값+2로 설정 해주었습니다.

길찾기 알고리즘



각 빨간점은 분기점을 의미하며
몬스터가 분기점에 도달 시 다음
분기점까지 갈 길을 선택합니다.

연결리스트를 활용하여 몬스터가
길의 분기점마다 방향을 설정 할
수 있도록 하였습니다.

	X좌표,z좌표		→		해당 분기점에서 갈 수 있는 다음 분기점들	
1	18,19	→	26,19	→	Current == destination 발생	
2	26,19	→	26,25	→	26,7	→ 18,19
3	26,25	→	5,25	→	26,32	→ 26,19
4	26,32	→	26,25	→	12,32	
5	5,25	→	26,25	→	5,40	
6	12,32	→	26,32	→	12,40	
7	12,40	→	12,32	→	5,40	
8	5,40	→	5,25	→	12,40	→ -1,40
9	-1,40	→	-1,27	→	5,40	→ -27,40
10	-1,27	→	-1,40	→		
11	26,7	→	26,19	→	5,7	→ 26,0
12	26,0	→	26,7	→	5,0	
13	5,0	→	-14,0	→	5,7	→ 26,0
14	5,7	→	5,0	→	-8,7	→ 26,7
15	-14,0	→	-27,0	→	-14,7	→ 5,0
16	-27,0	→	-14,0	→	-27,7	→
17	-14,7	→	-14,0	→	-27,7	→

-27,7	→	-14,7	→	-27,0	→
-27,14	→	-27,7	→	-14,14	→ -27,27
-8,7	→	5,7	→	-8,19	→
-8,19	→	-8,7	→	-14,19	→
-14,19	→	-8,19	→	-14,14	→
-14,14	→	-27,14	→	-14,19	→
-27,27	→	-7,27	→	-27,14	→ -27,40
-7,27	→	-27,27	→		
-27,40	→	-27,27	→	-1,40	→

```

if (current == destination) { //분기점 위치할때
    int num = pt.iscrossing(destination);
    std::list<Point> graph = pt.getListGraph(num);

    int selectList = rand() % (graph.size());

    std::list<Point>::iterator iter = graph.begin();
    for (int i = 0; i < selectList; i++) {
        iter++;
    }
    //목적지 재설정
    destination.x = iter->x;
    destination.z = iter->z;
}

```

Ex) iscrossing함수를 통해
현재 분기점의 행 index를 얻고
해당 행의 에서 랜덤함수를 통해
다음 분기점을 선택하였습니다.

감사합니다.